

تاسك Update Category — الشرح الكامل

خطوة بخطوة

ThreadMarket SaaS — التاسك مكتوب على Jira، وهنا هنشرح التاسك نفسه + تنفيذ الكود الكامل على أرض الواقع في المشروع.

1) شرح التاسك نفسه (Task Overview)

التاسك عبارة عن **Endpoint** جديد لتعديل (Update) كاتيجوري موجودة بتاعة براند معين. يعني كل براند له مجموعة categories، وده ال endpoint اللي بيسمح له يعدّل أي كاتيجوري من بتوعه.

المعلومة	التفاصيل
HTTP Method	PATCH
Endpoint	brands/{brandId}/categories/{catId}/
Authentication	مطلوب (Bearer Token)
Authorized Roles	brand فقط (مع إمكانية admin)
Path Parameters	brandId + catId
Request	جزء من بيانات الكاتيجوري (partial) — يعني ممكن ترسل اسم بس أو

{ success, message, data } مع 200

Success

Response

Validation Error (400) / Not Found (404) / Forbidden (403)

Error

Responses

ال Business Logic المطلوب:

• الكاتيجوري لازم تكون بتاعة البراند ده فعلاً (التأكد إن `catId.brandId === brandId`).

• لو مش بتاعته `Forbidden` أو `Not Found`.

• Side effects: **none** (مفيش حاجة تتغير غير الكاتيجوري نفسها).

ليه بنستخدم **PATCH** مش **PUT**؟ لأن ال **PATCH** يعني "تعديل جزئي" — بنسمح للمستخدم يبعث بس الحقول اللي عايز يعدلها، أما **PUT** فبيتوقع إنك تبعت الكيان كامل. التاسك بيقول "partial category" `payload` فده **PATCH** بالظبط.

(2) هيكلية المشروع (Structure) — فين الكود بيتكتب

المشروع منظم بنمط **Layered Architecture** (طبقات)، وكل موديول ليه 4 ملفات:

```
src/modules/category/
```

```
└─ category.routes.ts ← route تعريف ال middlewares
```

```
└─ category.controller.ts ← request استقبال ال response ورد ال
```

└─ category.service.ts ← (القرارات والأخطاء) business logic ال

└─ category.model.ts ← (Prisma) التعامل مع قاعدة البيانات

└─ category.validators.ts ← Zod ب input التحقق من ال

ملفات موجودة ومستخدم في المشروع (إحنا هنكمل عليها):

الوظيفة	الملف
يتحقق من ال JWT token ويحط ال user في req.user	src/shared/middleware/authenticate.ts
يتأكد إن ال role مسموح (مثلاً brand)	src/shared/middleware/authorize.ts
يتحقق من ال input ب Zod schema	src/shared/middleware/validate.ts
كلاس الخطأ الموحد (message + statusCode)	src/shared/errors/AppError.ts
يلفّ ال async controllers ويبعت الأخطاء ل error handler	src/shared/utils/asyncHandler.ts
بيتم تركيب كل ال routers — السطر 28 بيركب categoryRouter على api/v1/brands/	src/server.ts

معلومة مهمة عن الـ **route**: في `server.ts` السطر 28، الـ `categoryRouter` متشال على

`category.routes` الـ `route` يعني أي `app.use("/api/v1/brands", categoryRouter)` جوه

هيتضاف بعد `api/v1/brands/` فلو الـ `route` في الملف هو `"brandId/categories/:catId/"`، فـ

الـ `endpoint` الكامل هيكون: `PATCH /api/v1/brands/:brandId/categories/:catId`

3) الـ `Data Model (schema.prisma)` — شكل الكاتيجوري في

الداتابيز

```
model Category {  
  
  id          String    @id @default(uuid()) @db.Uuid  
  
  brandId     String    @map("brand_id") @db.Uuid  
  
  name        String    @map("category_name")  
  
  imageUrl    String?   @map("image_url")  
  
  media       Json  
  
  ishidden    Boolean   @default(false)  
  
  Products    Product[]  
  
  createdAt   DateTime  @default(now())  
}
```

```

updateAt DateTime @default(now())

brand Brand @relation(fields: [brandId], references: [id], onDelete:
Cascade)

@@unique([brandId, name]) // نفس الاسم ممنوع يتكرر لنفس البراند

@@map("categories")

}

```

الحقول اللي ممكن نعدّلها في التاسك ده: `name` و `imageUrl` و `ishidden` ال `brandId` ممنوع يتغير (الكاتيجوري مش هتتحول لبراند تاني).

نقطة الانتباه: جدول `Category` فيه `@@unique([brandId, name])` — يعني لو حبيت تعدّل الاسم، لازم نتأكد إن الاسم الجديد مش متاخذ من كاتيجوري تانية لنفس البراند، وإلا هنشيل خطأ `Conflict` من الداتابيز نفسه. ال `service` هيتعامل مع ده.

4) التنفيذ خطوة بخطوة (Implementation)

هنا هننقّذ ال `Update` خطوة بخطوة، بنفس نمط ال `Create` الموجود في المشروع. الترتيب: ال `Model` → ال `Service` → ال `Controller` → ال `Validator` → ال `Route`.

إضافة دوال الداتابيز — `src/modules/category/category.model.ts` تعديل

نضيف دالة `findByIdWithinBrand` (للتأكد إن الكاتيجوري بتاعة البراند) ودالة `update` (للتعديل الفعلي):

```
import prisma from "../../config/database";
```

```
class CategoryModel {
```

```
    // موجوده قبل كده — لفحص تكرار الاسم
```

```
    async findByNameWithinBrand(brandId: string, name: string) {
```

```
        return prisma.category.findFirst({
```

```
            where: { brandId, name },
```

```
            select: { id: true },
```

```
        });
```

```
    }
```

```
    // جلب كاتيجوري بشرط أنها تخص براند معين (جديد)
```

```
    async findByIdWithinBrand(brandId: string, catId: string) {
```

```
        return prisma.category.findFirst({
```

```
            where: { id: catId, brandId },
```

```
});
```

```
}
```

```
// تعديل جزئي — بنبعت فقط الحقول اللي اتغيرت (جديد)
```

```
async update(catId: string, data: { name?: string; imageUrl?:  
string; ishidden?: boolean }) {
```

```
  return prisma.category.update({
```

```
    where: { id: catId },
```

```
    data: {
```

```
      ...(data.name !== undefined ? { name: data.name } : {}),
```

```
      ...(data.imageUrl !== undefined ? { imageUrl:  
data.imageUrl } : {}),
```

```
      ...(data.ishidden !== undefined ? { ishidden:  
data.ishidden } : {}),
```

```
      updateAt: new Date(),
```

```
    },
```

```
    select: { id: true },
```

```
});
```

```
}
```

```
// موجوده قبل كده
```

```
async create(brandId: string, name: string, imageUrl?: string) {
```

```
  return prisma.category.create({
```

```
    data: {
```

```
      brandId,
```

```
      name,
```

```
      media: {},
```

```
      ...(imageUrl ? { imageUrl } : {}),
```

```
    },
```

```
    select: { id: true },
```

```
  });
```

```
}
```



```
}
```

```
export const categoryModel = new CategoryModel();
```

شرح كل جزء:

• `findByIdWithinBrand`: يستخدم `findFirst` بـ `catId`, `where`: { `id`: `catId`, `brandId` — ده هو التطبيق العملي لـ "Validation Rules: category must belong to"

• `update`: في Prisma الـ `update` بيرمي خطأ لو الـ `id` مش موجود، وبنبني الـ `data` بشكل

ديناميكي عشان نعدّل الحقول المبعوثة بس (`updateAt`: `new Date`.(`partial update`)).

عشان نحدّث طابع الوقت.

• استخدمنا `spread` مع (`...(!== undefined ? {...} : {})`) `check` لأن

كده لو المستخدم مبعتهاش حاجة، الحقل مش هيتعدل.

تعديل `src/modules/category/category.service.ts` — **Business Logic**

نضيف ميثود `updateCategory` اللي فيها كل القرارات:

```
import AppError from "../../shared/errors/AppError";
```

```
import { StatusCodes } from "http-status-codes";
```

```
import { brandModel } from "../brand/brand.model";
```

```
import { categoryModel } from "./category.model";
```

```
class CategoryService {

    // موجودة قبل كده

    async createCategory(brandId, userId, role, name, imageUrl?) {

        // نفس اللي موجود ...

    }

    // تعديل كاتيجوري (جديد)

    async updateCategory(brandId: string, catId: string, userId: string,
        role: string, data: { name?: string; imageUrl?: string; ishidden?:
        boolean }) {

        // نتأكد إن البراند موجود (1)

        const brand = await brandModel.findBrandWithDocuments(brandId);

        if (!brand) {

            throw new AppError("Brand not found",
                StatusCodes.NOT_FOUND);
```

```
}
```

```
// 2) بس admin صلاحية: صاحب البراند أو
```

```
if (brand.userId !== userId && role !== "admin") {
```

```
    throw new AppError("You are not authorized to update this  
category", StatusCodes.FORBIDDEN);
```

```
}
```

```
// 3) الكاتيجوري موجودة وبتاعة البراند ده؟
```

```
const category = await  
categoryModel.findByIdWithinBrand(brandId, catId);
```

```
if (!category) {
```

```
    throw new AppError("Category not found or does not belong to  
this brand", StatusCodes.NOT_FOUND);
```

```
}
```

```
// 4) لو المستخدم بيعدّل الاسم، نتأكد إنه مش مكرر لنفس البراند
```

```
        if (data.name && data.name !== category.name) {

            const existing = await
categoryModel.findByNameWithinBrand(brandId, data.name);

            if (existing && existing.id !== catId) {

                throw new AppError("Category name already exists in this
brand", StatusCodes.CONFLICT);

            }

        }

    }

    // 5) التعديل الفعلي

    const updated = await categoryModel.update(catId, data);

    return { id: updated.id };

}

}

export const categoryService = new CategoryService();
```

شرح كل خطوة في ال service:

• **الخطوة 1:** البراند نفسه لازم موجود — لو لا Not Found 404.

• **الخطوة 2:** التأكد إن المستخدم هو صاحب البراند أو admin → لو لا Forbidden 403. ده

اللي يطلبه التاسك في "Authorized Roles: brand".

• **الخطوة 3:** ده أهم سطر في التاسك كله — بنستخدم findByIdWithinBrand اللي بيعمل شرط

مزدوج (id + brandId) في query واحدة. كده بنتأكد إن الكاتيجوري موجودة وإنها بتاعة البراند ده في

نفس الوقت → لو لا 404.

• **الخطوة 4:** لو الاسم بيتعدل، بنتحقق إن مفيش كاتيجوري تانية لنفس البراند عندها نفس الاسم

الجديد (except الكاتيجوري نفسها) → لو في Conflict 409. ده بيحمي ال

unique([brandId, name])@@ من الداتابيز.

• **الخطوة 5:** بننادي ال model اللي بيعمل ال Prisma update ويرجع ال id.

Request/Response ال — src/modules/category/category.controller.ts تعديل

نضيف ال controller اللي بياخد ال params وال body ويستدعي ال service:

```
import { Request, Response } from "express";

import { categoryService } from "../category.service";

import { StatusCodes } from "http-status-codes";

import { asyncHandler } from "../../../shared/utils/asyncHandler";

import AppError from "../../../shared/errors/AppError";
```

```
export const createCategory = asyncHandler(async (req: Request, res:
Response) => {

    // ... الموجود قبل كده

});

// تعديل كاتيجوري (جديد)

export const updateCategory = asyncHandler(async (req: Request, res:
Response) => {

    const { brandId, catId } = req.params as { brandId: string; catId:
string };

    const { name, imageUrl, ishidden } = req.body;

    const user = (req as any).user;

    if (!user?.id) {

        throw new AppError("Not authenticated",
StatusCodes.UNAUTHORIZED);
```

```
}

const result = await categoryService.updateCategory(

  brandId, catId, user.id, user.role,

  { name, imageUrl, ishidden }

);

res.status(StatusCode.OK).json({

  success: true,

  message: "Category updated successfully",

  data: result,

});

});
```

شرح:

• بنقرأ `brandId` و `catId` من `req.params` (عشان هما في ال URL).

• بنقرأ `name`, `imageUrl`, `ishidden` من ال body — كلهم optional عشان "

payload". لو المستخدم مبعتهاش حاجة هيبقوا undefined وال service هيتجاهلهم.

ال user جاي من ال authenticate middleware اللي حطه في req.user.

ال 200 OK response بنفس شكل ال JSON المطلوب في التاسك بالظبط.

Zod ب Validation ال — src/modules/category/category.validators.ts تعديل

نضيف schema للتعديل (كل الحقول optional لأنها partial):

```
import { z } from "zod";

export const createCategorySchema = z.object({

  brandId: z.string().uuid({ message: "Invalid brand ID format" }),

  name: z.string().min(1, "Category name is required"),

  imageUrl: z.string().url("Invalid image URL format").optional(),

});

// للتعديل الجزئي Schema (جديد)

export const updateCategorySchema = z.object({
```



```

    brandId: z.string().uuid({ message: "Invalid brand ID format" }),

    catId: z.string().uuid({ message: "Invalid category ID format" }),

    name: z.string().min(1, "Category name cannot be empty").optional(),

    imageUrl: z.string().url("Invalid image URL format").optional(),

    ishidden: z.boolean().optional(),

  });

```

شرح:

• `req.params + req.body` في المشروع بدمج `validate middleware` الـ

`req.query` مع بعض، فبنضع `brandId` و `catId` (اللي جايين من الـ `params`) في نفس الـ `schema`

• `optional()` على كل حقول الـ `body` عشان المستخدم ممكن يعدل حقل واحد بس — وده اللي قصده التاسك بـ "partial category payload".

• لو حاجة غلط، الـ `middleware` هيرمي `Validation Error 400` بأول رسالة خطأ.

ربط كل حاجة — `src/modules/category/category.routes.ts` تعديل

نضيف الـ `route` الجديد بالـ `middlewares` بالترتيب الصحيح:

```

import { Router } from "express";

import { validate } from "../../shared/middleware/validate";

```

```
import authenticate from "../../shared/middleware/authenticate";

import { authorize } from "../../shared/middleware/authorize";

import { createCategory, updateCategory } from "../category.controller";

import { createCategorySchema, updateCategorySchema } from
"./category.validators";


const router = Router();


router.post("/:brandId/categories",

    authenticate,

    authorize("brand"),

    validate(createCategorySchema),

    createCategory,

);


// تعديل كاتيجوري (جديد)
```

```

router.patch("/:brandId/categories/:catId",

    authenticate,          // 1) login لازم يكون عامل

    authorize("brand"),    // 2) role لازم ال brand يكون

    validate(updateCategorySchema), // 3) تحقق من المدخلات

    updateCategory,        // 4) ال controller

);

export default router;

```

شرح ترتيب ال middlewares:

بيأكد إيه؟	Middleware	
فيش user؟ → لو لأ: 401 Unauthorized	authenticate	1
ال role بتاعته brand؟ → لو لأ: 403 Forbidden	authorize("brand")	2
المدخلات صح؟ → لو لأ: 400 Validation Error	validate(updateCategorySchema)	3
المنطق والرد: 200 أو 403/404	updateCategory	4

URL الكامل: PATCH /api/v1/brands/:brandId/categories/:catId — لإن ال

router مرّّب على api/v1/brands/ في server.ts.

5) اختبار ال Endpoint (Testing)

مثال بطلبه كامل بال <token :<Authorization: Bearer

```
PATCH http://localhost:PORT/api/v1/brands/<brandId>/categories/<catId>
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

```
// Body (جزء من الحقول فقط) :
```

```
{  
  
  "name": "Eid Collection 2026"  
  
}
```

الرد المتوقع (success):

```
{  
  
  "success": true,
```

```
"message": "Category updated successfully",

"data": {

  "id": "5f2a8e6c-3b11-4c5e-9d2a-1f0b3c4d5e6f"

}

}
```

6) جدول الـ Error Codes اللي ممكن تحصل

سببها فين في الكود	Status Code	الحالة
Zod في الـ UUID — validate middleware غلط، اسم فاضي، URL مش صحيح	400	Validation Error
authenticate — مفيش token أو token غير صالح	401	Unauthorized
authorize (role مش brand) أو service (مش صاحب البراند)	403	Forbidden
service — البراند مش موجود أو الكاتيجوري مش موجودة / مش بتاعة البراند	404	Not Found
service — الاسم الجديد مكرر لنفس البراند	409	Conflict

معلومة مهمة عن الـ 404: التاسك يقول "Validation Rules: category must belong to the brand".

في التطبيق عملناها بـ `findByIdWithinBrand` اللي بيدور بـ `{ id: catId, brandId }` في `query` واحدة. ده أفضل من إننا نجيب الكاتيجوري أولاً وبعدين نقارن — لأنه بيقلل عدد الـ `database queries` ولأنه بيمنع تسريب معلومة إن الكاتيجوري موجودة أصلاً (بيحمي من الـ `enumeration attacks`).

(7) المخطط النهائي (Flow Diagram)

الطلب: `PATCH /brands/{brandId}/categories/{catId}`

|

▼

[1] `authenticate` — 401 خطأ Unauthorized

|

▼

[2] `authorize("brand")` — 403 — Forbidden

|

▼

[3] `validate(updateCategorySchema)` — 400 — Validation Error

|

▼

[4] updateCategory (Controller)

|

▼

[5] updateCategory (Service)

|

└─ brand 404 لا يوجد? Not Found

└─ brand.userId === userId أو admin? — 403 لا Forbidden

└─ category brand? موجودة وبتاعة — 404 لا Not Found

└─ name 409 (الاسم موجود) Conflict

▼

[6] categoryModel.update() (Prisma UPDATE)

▼

[7] 200 { success: true, message: "Category updated successfully",
data: { id } }

(8) ملخص سريع (Checklist)

- أضفنا `findByIdWithinBrand + update` في الـ `model`
- أضفنا `updateCategory` في الـ `service` (وجود البراند → الصلاحية → ملكية الكاتيجوري → منع التكرار)
- أضفنا `updateCategory` في الـ `controller` (يرد 200 بنفس شكل JSON المطلوب)
- أضفنا `updateCategorySchema` بـ `Zod` (كل الحقول `optional = partial`)
- أضفنا `router.patch("/:brandId/categories/:catId", ...)` بالـ `middlewares` بالترتيب
- كل الأخطاء عبر `error handler` → `AppError` الموحد في `server.ts`